A.E.D. Laboratorio 2

Guillermo Román
guillermo.roman@upm.es

Julio García
juliomanuel.garcia@upm.es

Lars-Åke Fredlund
lfredlund@fi.upm.es

Tonghong Li
tonghong@fi.upm.es

Manuel Carro
mcarro@fi.upm.es

Francisco de Borja Lozano
francisco.lozano@upm.es

Normas

- Fechas de entrega y penalización:

Hasta el Martes 13 de Octubre, 16:00 horas	0 %
Hasta el Miércoles 14 de Octubre, 16:00 horas	20 %
Hasta el Jueves 15 de Octubre, 16:00 horas	40 %

  Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Tarea: implementar la clase `Polinomio`

Se quiere desarrollar un librería para poder manipular y evaluar polinomios sobre una variable y con coeficientes y exponentes enteros.

Recordatorio:

- Un *monomio* es una expresion algebraica, por ejemplo $5x^2$, compuesto por un coeficiente (5), una variable o parte literal (x), y un exponente o grado (2).
- Un *polinomio* es una suma de monomios, por ejemplo $5x^2 + 3x + 81$.
- La librería trabajará con polinomios de una variable (x) y con coeficientes y exponentes enteros. El valor del exponente siempre es mayor o igual que cero (0).
- Dado que solo usamos una variable, no necesitamos representarla y solo es preciso guardar el coeficiente y exponente de cada monomio.

Representación de un polinomio

- Dentro la clase representamos un polinomio como una lista de monomios (de tipo `PositionList`) guardada en el atributo `terms` dentro `Polinomio`:

```
PositionList<Monomio> terms;
```

- Condiciones a cumplir:
  - Los monomios deben estar ordenados en orden decreciente de su grado. Por ejemplo, el polinomio $x^5 + -2x^3 + 8$ se representa como

    `[Monomio(1,5), Monomio(-2,3), Monomio(8,0)]`
  - No puede haber varios monomios del mismo grado en la lista. P.e., la representación `[Monomio(1,5), Monomio(2,5)]` no es válida.
  - No puede haber ningún monomio con coeficiente 0 en la lista. El polinomio 0 (o polinomio nulo) se representa como la lista vacía.

Representación de un monomio

- Un monomio se representa por un par de un coeficiente y un exponente:

```
public class Monomio extends Pair<Integer,Integer> {

  public Monomio(int coeficiente, int exponente) {
    super(coeficiente, exponente);
  }

  public int coeficiente() { return left(); }
  public int exponente() { return right(); }
}
```

Métodos a implementar en la clase Polinomio

```
// suma dos polinomios, subtraccion y multiplicacion
static Polinomio suma(Polinomio p1, Polinomio p2);
static Polinomio resta(Polinomio p1, Polinomio p2);
static Polinomio multiplica(Monomio m, Polinomio p);
static Polinomio multiplica(Polinomio p1, Polinomio p2);

// devuelve el grado del polinomio
int grado();

// devuelve la derivada de un polinomio
Polinomio derivada();

// Devuelve un polinomio nuevo donde x = p2. Se sustituye el variable
// x por el polinomio p2 dentro el polinomio p1
Polinomio componer(Polinomio p1, Polinomio p2);

// comprueba si dos polinomios son iguales
boolean equals(Object obj);
```

- Documentación detallada en

https://costa.ls.fi.upm.es/teaching/aed/docs/practicas/polinomios/

Tarea opcional: implementar un *parser* para polinomios

- Podéis obtener 2 puntos adicionales (para un máximo de 12 puntos) implementando correctamente el constructor

  ```
  public Polinomio(String polinomio) { ... }
  ```

  que inicializa el atributo `terms` dentro `Polinomio` con el polinomio representado por el `String polinomio` .
- Textualmente un polinomio esta representado por un suma de monomios separados por '+' .
- Cada monomio puede tener una de las formas:

  `Kx^N` `x^N` `Kx` `x` `K`

  con `N` y `K` enteros, $\texttt{N} \geq 0$ y $\texttt{K} \neq 0$ **excepto** si es el único monomio en el polinomio, en cuyo caso puede ser $\texttt{K} = 0$.
- Pueden aparecer espacios ' ' entre las diferentes partes.
- Suponed que los monomios en el `String` aparecen en orden descendente de exponentes y que nunca hay dos monomios con exponentes iguales.

Ejemplos del uso del constructor opcional

```
new Polinomio("2x^3 + 9")    // == 2x^3 + 9x^0
                             // ==> [Monomio(2,3), Monomio(9,0)]
new Polinomio("2x^3 + -9")   // == 2x^3 + -9x^0
                             // ==> [Monomio(2,3), Monomio(-9,0)]
new Polinomio("x")           // == 1x^1 ==> [Monomio(1,1)]
new Polinomio("5")           // == 5x^0 ==> [Monomio(5,0)]
new Polinomio("8x")          // == 8x^1 ==> [Monomio(8,1)]
new Polinomio("0")           // representado con lista vacia
                             // ==> []
```

Ejemplos

```
Polinomio p1 = new Polinomio("2x^2 + 9x + 3");
Polinomio p2 = new Polinomio("8x^2 + -7");
Polinomio.suma(p1,p2);    // => 10x^2 + 9x^1 + -4
Polinomio.resta(p2,p1);   // => 6x^2 + -9x^1 + -10

Polinomio.multiplica(p1,new Polinomio("5")); // => 10x^2 + 45x^1 + 15
Polinomio.multiplica(p1,p2); // => 16x^4 + 72x^3 + 10x^2 + -63x^1 + -21

p1.grado();               // => 2
p1.derivada();            // => 4x^1 + 9

Polinomio p3 = new Polinomio("x + 2");
Polinomio.componer(p1,p3);  // Sustituyendo x por p3 en p1
                            // => 2*(x + 2)^2 + 9*(x + 2) + 3
                            // = 2x^2 + 17x^1 + 29


Polinomio.componer(p1,new Polinomio("5"));
// => 98 (sustituyendo x por 5 en p1)

p1.equals(p2);                                    // => false
new Polinomio("5x").equals(new Polinomio("5x"));  // => true
```

Consejos

- Hay mucho material online sobre las operaciones basicas en polinomios, consulta por ejemplo `https://es.wikipedia.org/wiki/Polinomio` y `https://es.wikipedia.org/wiki/Monomio`.
- Para implementar el método `componer` es aconsejable implementar un método (auxiliar) para elevar un polinomio a un exponente entero (≥ 0).
- No sólo está permitido, sino recomendado, definir y usar métodos auxiliares para reducir la cantidad de código. Por ejemplo, las operaciones de sumar y restar son muy parecidas y es posible implementar un método auxiliar usado por `suma` y `resta`.

Consejos sobre la implementación del constructor opcional

Consulta la documentación estándar de Java para saber más sobre las operaciones sobre `String`. Métodos útiles (`s` es un `String`):

- `s.split(regexp)` devuelve un *array* de `String` donde cada elemento ha sido separado por `regexp`. Por ejemplo, si `s = "2x^3 + 1x + 2"` entonces `s.split("\\+")` devuelve el *array* `["2x^3 ", " 1x ", " 2"]`. Otros ejemplos:

```
"xA".split("x")  => ["", "A"]
"Ax".split("x")  => ["A"]
"x".split("x")   => []
"abc".split("x") => ["abc"]
```

- Un "regexp" es una expresion regular. Para esta tarea solo hace falta reconocer caracteres individuales. Por ejemplo la expresion regular `"x"` reconoce el caracter `'x'`. Sin embargo, algunas caracteres como '+' y '^' tiene signicado especial. Para reconocer estos caracteres tenemos que declarar que no nos interesa el signficado especial. Usaremos `"\\+"` y `"\\^"` para eso.

Consejos sobre la implementación del constructor opcional

- `s.contains(s1)` comprueba si `s1` aparece dentro `s`. Por ejemplo: `new String("hello").contains("lo")` devuelve `true`.
- Para convertir un string representando un número a un entero se puede llamar `Integer.parseInt(s)`. Por ejemplo `Integer.parseInt("-24")` devuelve el entero -24.
- `s.trim()` devuelve un nuevo `String` donde los espacios al principio y final de `s` están eliminados. Por ejemplo, `new String(" hola ").trim()` devuelve `"hola"`.

Evaluación

Seguir estos consejos os permitirá conseguir mejores resultados!

- Corrección
- Ausencia de código repetido con la misma funcionalidad (podéis usar métodos auxiliares para evitarlo)
- Concisión del código
- Legibilidad, incluida selección de nombres descriptivos para variables y métodos
- El código debe estar correctamente indentado y con comentarios *útiles* cuando lo veáis necesario
- Eficiencia:
  - Se valorará la complejidad computacional del código
  - Se valorará no iterar innecesariamente en los recorridos de las estructuras de datos

Notas

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test `TesterLab2` correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que el método sea correcto (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final